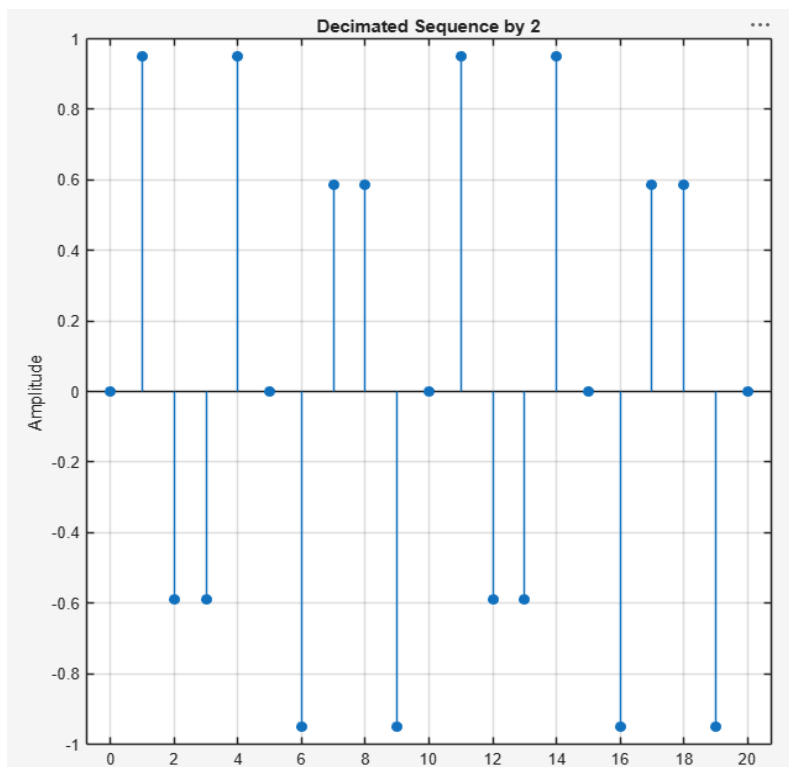
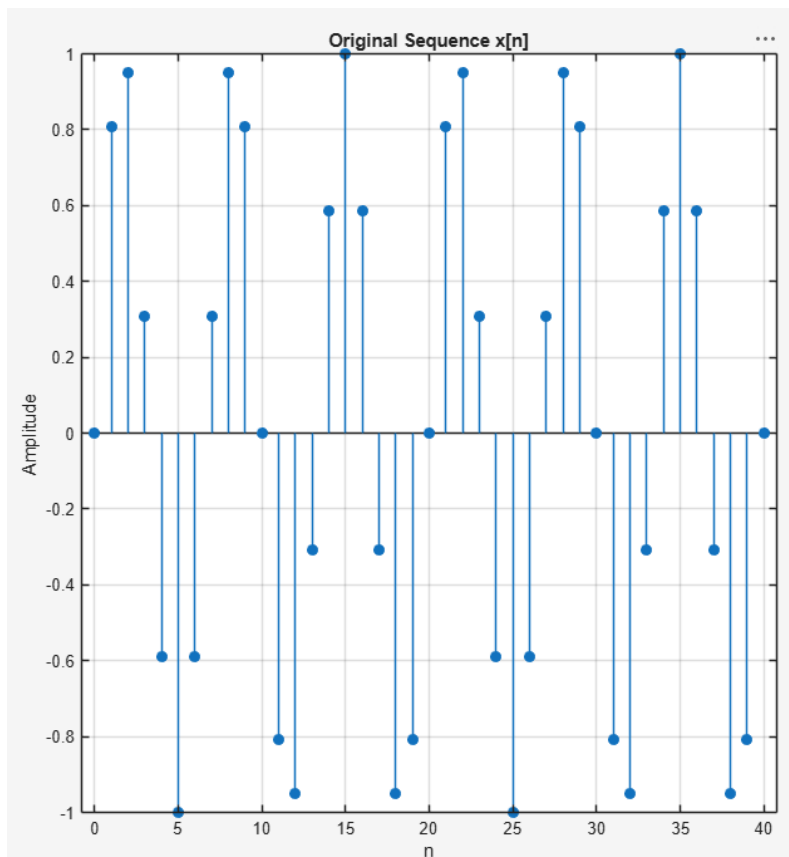


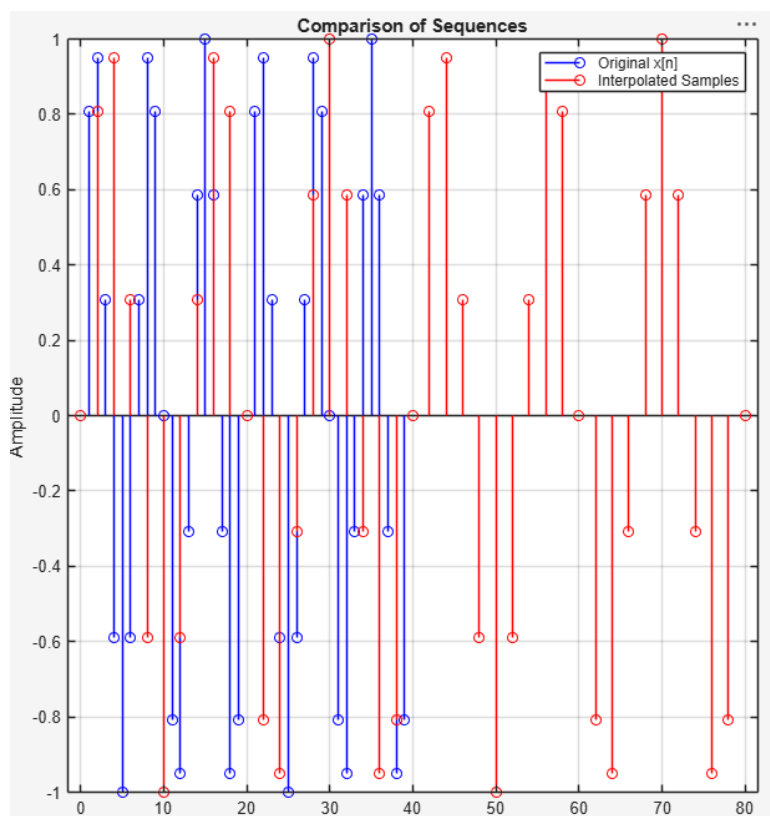
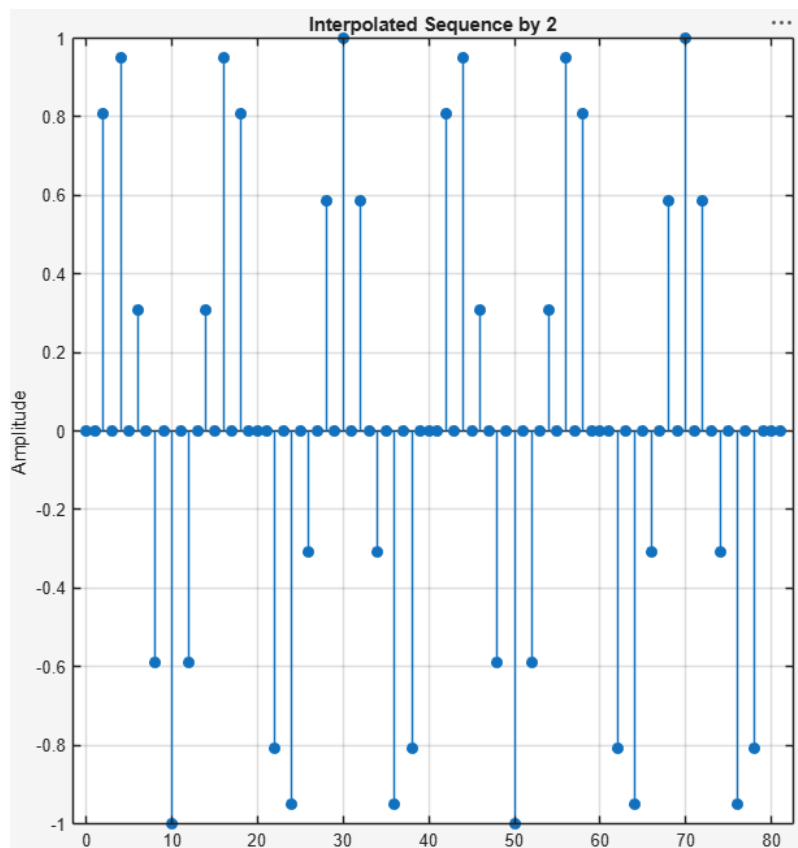
8.1 Implementation and Analysis of Decimation and Interpolation in Multirate Signal Processing Using MATLAB

/MATLAB Drive/Dsp_8a.m

```
2 clear;
3 close all;
4 % Original Sequence
5 n = 0:40;
6 x = sin(0.3*pi*n);
7 % Decimation by 2
8 y = x(1:2:end);
9 nd = 0:length(y)-1;
10 % Interpolation by 2
11 v = zeros(1,2*length(x));
12 v(1:2:end) = x;
13 ni = 0:length(v)-1;
14 % Figure 1: Original Sequence
15 figure;
16 stem(n,x,'filled');
17 grid on;
18 title('Original Sequence x[n]');
19 xlabel('n');
20 ylabel('Amplitude');
21 % Figure 2: Decimated Sequence
22 figure;
23 stem(nd,y,'filled');
24 grid on;
25 title('Decimated Sequence by 2');
26 xlabel('n');
27 ylabel('Amplitude');
28 % Figure 3: Interpolated Sequence
29 figure;
30 stem(ni,v,'filled');
31 grid on;
32 title('Interpolated Sequence by 2');
33 xlabel('n');
34 ylabel('Amplitude');
35 % Figure 4: Comparison
36 figure;
37 stem(n,x,'b'); hold on;
38 stem(0:2:length(v)-2,v(1:2:end),'r');
39 legend('Original x[n]','Interpolated Samples');
40 grid on;
41 title('Comparison of Sequences');
42 xlabel('n');
43 ylabel('Amplitude');
```

Output:





8.2 Polyphase Implementation of Decimation and Interpolation Filters Using MATLAB

/MATLAB Drive/Dsp_8b.m

1	clc;	
2	clear;	
3	close all;	
4		
5	%% Input Signal	
6	Fs = 1000; % Sampling Frequency	
7	t = 0:1/Fs:1; % Time Vector	
8	% Test Signal: 50 Hz and 120 Hz sine waves	
9	x = sin(2*pi*50*t) + 0.5*sin(2*pi*120*t);	
10		
11	%% FIR Low-Pass Filter Design	
12	N = 20; % Filter Order	
13	Fc = 0.4; % Normalized Cutoff Frequency (relative to Nyquist)	
14	h = fir1(N, Fc, hamming(N+1));	
15		
16	%% Decimation Factor	
17	M = 2;	
18	% Polyphase Decimation (Filter then Downsample by M)	
19	y_dec = upfirdn(x, h, 1, M);	
20		
21	%% Interpolation Factor	
22	L = 2;	
23	% Polyphase Interpolation (Upsample by L then Filter)	
24	y_int = upfirdn(y_dec, h, L, 1);	
25		
26	%% Compensate for FIR Group Delay	
27	% Delay introduced by two passes through order N filter = N samples	
28	delay = N;	
29		
30	% Align reconstructed signal with original signal	
31	if length(y_int) > delay	
32	y_int_aligned = y_int(delay + 1:end);	
33	else	
34	y_int_aligned = y_int;	
35	end	
36		
37	% Truncate or pad vectors to match lengths for direct comparison	
38	min_len = min(length(x), length(y_int_aligned));	
39	x_comp = x(1:min_len);	
40	y_comp = y_int_aligned(1:min_len);	
41		

```

42  %% Time Axes
43  t1 = (0:length(x)-1)/Fs;
44  t2 = (0:length(y_dec)-1)/(Fs/M);
45  t_comp = (0:min_len-1)/Fs;
46
47  %% BEST GRAPH OUTPUT
48  figure('Name','Polyphase Decimation and Interpolation','NumberTitle','off');
49
50  % 1. Original Signal
51  subplot(4,1,1)
52  plot(t1, x, 'b', 'LineWidth', 1.5)
53  title('Original Input Signal')
54  xlabel('Time (s)')
55  ylabel('Amplitude')
56  grid on
57
58  % 2. Decimated Signal
59  subplot(4,1,2)
60  stem(t2, y_dec, 'r', 'filled', 'MarkerSize', 3)
61  title('Decimated Signal (M = 2)')
62  xlabel('Time (s)')
63  ylabel('Amplitude')
64  grid on
65
66  % 3. Interpolated Signal
67  subplot(4,1,3)
68  plot((0:length(y_int)-1)/Fs, y_int, 'k', 'LineWidth', 1.5)
69  title('Interpolated/Reconstructed Signal (L = 2, Unaligned)')
70  xlabel('Time (s)')
71  ylabel('Amplitude')
72  grid on
73
74  % 4. Comparison (Delay Compensated)
75  subplot(4,1,4)
76  plot(t_comp, x_comp, 'b', 'LineWidth', 1.5)
77  hold on
78  plot(t_comp, y_comp, 'r--', 'LineWidth', 1.5)
79  title('Comparison of Original and Reconstructed Signals (Delay Compensated)')
80  xlabel('Time (s)')
81  ylabel('Amplitude')
82  legend('Original Signal', 'Reconstructed Signal')
83  grid on
84
85  %% Display Information
86  disp('-----');
87  disp('POLYPHASE DECIMATION & INTERPOLATION');
88  disp('-----');
89  fprintf('Original Signal Length      = %d\n', length(x));
90  fprintf('Decimated Signal Length      = %d\n', length(y_dec));
91  fprintf('Interpolated Signal Length    = %d\n', length(y_int));
92
93  %% Separate Figure for Better Visualization
94  figure('Name', 'Overlay Comparison', 'NumberTitle', 'off');
95  plot(t_comp, x_comp, 'b', 'LineWidth', 2)
96  hold on
97  plot(t_comp, y_comp, 'r--', 'LineWidth', 2)
98  title('Original vs Reconstructed Signal (Aligned)')
99  xlabel('Time (s)')
100 ylabel('Amplitude')
101 legend('Original Signal', 'Reconstructed Signal')
102 grid on

```

Output:

